Faculdade de Ciências da Universidade de Lisboa

Break-it Phase Vulnerability Analysis

Secure Gallery Log Systems

José Sampaio, fc66191
Rodrigo Neto, fc59850
Gabriel Cambraia, 58671

Segurança de Redes e Sistemas Distribuídos
2025/2026

Contents

1 Introduction

This document reports our vulnerability analysis for the Break-it phase of the SRSD Gallery Log project. We were assigned two independent implementations to examine: Group 1 and Group 21. For each implementation, we document how we built and ran the submitted programs, what parts of the security design we reviewed, which attacks we attempted, and whether we found a reproducible security vulnerability.

According to the project statement, valid Break-it findings include crashes, integrity violations, and confidentiality violations. We therefore distinguish security-relevant vulnerabilities from ordinary correctness bugs. For each successful attack, we provide a high-level summary, the root cause, reproduction steps, the expected behavior of a correct implementation, the observed vulnerable behavior, and references to any attack code or testcase files submitted alongside this report.

2 Evaluation Criteria Used

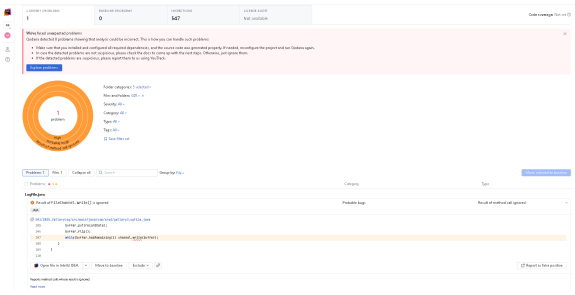
We evaluated each assigned implementation against the following security properties from the project specification:

- **Crash resistance:** malformed or adversarial log files should not cause abnormal termination; they should be rejected cleanly.
- **Integrity:** an attacker without the token should not be able to modify, delete, insert, reorder, or forge log entries so that `logread` accepts the resulting file.
- **Confidentiality:** an attacker without the token should not be able to infer private log contents or query results from the file representation alone.
- **Authentication:** commands using the wrong token should not be able to append to or read an existing log.
- **State consistency:** invalid gallery transitions should be rejected and should not leave the log in a partially modified state.

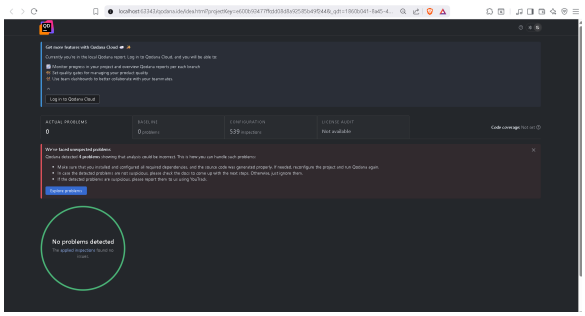
The attack templates below follow the testcase format requested in the project statement: create or obtain a valid log, perform the adversarial step without the token, and then show how the target implementation processes the resulting file incorrectly.

3 Static Analysis with Qodana

As an additional preliminary step, we ran JetBrains Qodana as a static-analysis tool over the assigned implementations. The goal was to identify automatically detectable security issues before performing manual code review and behavioral testing. Qodana did not report any security-relevant findings for either assigned project, so the vulnerabilities and behavioral deviations discussed later in this report were identified through manual analysis and targeted testing instead.



Group 1 Qodana report



Group 21 Qodana report

Figure 1: Static-analysis reports produced with Qodana. No security issues were reported by the tool.

4 Methodology: Shared Behavioral Test Suite

Before analyzing the assigned implementations, we developed one shared behavioral test suite and adapted only the wrapper commands for each target. Since the Group 1 and Group 21 demos exercise equivalent behavior, we do not include the full terminal transcripts in this report. Instead, we summarize what the demos test once in Table 1, and then report the relevant deviations for each group in their respective sections. The complete scripts and raw outputs are submitted as external artifacts.

Category	**Behavior tested**
Basic gallery flow	Employee and guest arrivals and departures at gallery level.
Room transitions	Entering rooms, leaving rooms, and preventing impossible room occupancy states.
Query behavior	Current state queries (`-S`), room-history queries (`-R`), and intersection queries (`-I`).
Invalid transitions	Duplicate arrivals, leaving without entering, leaving a room before entering it, timestamp regressions, invalid names, and conflicting flags.
Token handling	Rejection of operations attempted with an incorrect token.
Tamper handling	Rejection of logs modified directly on disk after valid creation.
Batch mode	Processing a batch file with valid lines and intentionally invalid lines while preserving safe log state.

Table 1: Shared behavioral coverage used for both assigned implementations

This condensed presentation keeps the report focused on vulnerability analysis rather than terminal output. For each group, we classify each behavioral deviation as either a security-relevant failure or a correctness/compatibility issue. In particular, incorrect exit codes are only treated as security issues when the unsafe action is actually accepted or the protected data is exposed.

5 Implementation G21: Group 21

5.1 Target Summary

- **Assigned implementation:** Group 21.
- **Language and build system:** C with Makefile and OpenSSL.
- **Programs analyzed:** `logappend` and `logread`.
- **Behavioral status:** Passed most functional and security-behavior tests.
- **Main result:** The implementation behaved safely in the tested invalid-action cases, but it reported some safe rejections with the wrong return code. It also showed the same room-history duplication issue observed in Group 1.

5.2 How We Built and Ran the Implementation

Observed result: The implementation compiled successfully using GCC and OpenSSL on Ubuntu Linux. Both `logappend` and `logread` executed correctly during normal operation. No special setup beyond OpenSSL development libraries was required.

```
cd group21
make
./logappend -T 1 -K testToken -E Alice -A test.log
./logread -K testToken -S test.log
```

5.3 Behavioral Test Results

Table 2 summarizes the Group 21 execution of the shared test suite described in Section 4. The full raw output is omitted from the main text because it mostly consists of successful setup commands and repeated expected outputs.

Test area	Result	Relevant observation
Basic gallery flow	Pass	Employee and guest arrivals/departures were reflected correctly by `-S`.
Room transitions	Pass	Room entry and exit transitions were enforced safely.
State and intersection queries	Pass	`-S` and `-I` returned the expected state for the tested scenarios.
Room history query	Minor issue	Revisited rooms were printed again, e.g., `3,7,3` instead of the deduplicated `3,7`.
Invalid actions	Safe with return-code issue	Unsafe operations were not applied, but some were signalled with an incorrect return code.
Wrong token and tampering	Pass	The tested unsafe operations did not expose protected data or produce an accepted corrupted state.
Batch mode	Pass	Valid lines were processed and intentionally invalid lines were handled safely.

Table 2: Condensed behavioral results for Group 21

5.4 Behavioral Findings

5.4.1 Room History Duplication

The `-R` query includes repeated room visits in the output. For example, if a person visits room 3, later room 7, and then room 3 again, the implementation prints `3,7,3`. Our interpretation of the expected behavior is that room history should list each visited room once, in first-visit order, i.e., `3,7`. This is a correctness issue rather than a direct security vulnerability because it does not let an attacker read, modify, or forge protected log contents.

5.4.2 Incorrect Return Codes for Safe Rejections

Some invalid operations were rejected safely but did not use the expected error signalling convention. In these cases, the important security property still held: the invalid action was not committed to the log. Therefore, we classify these as compatibility/correctness issues, not successful Break-it attacks.

5.5 Reconnaissance and Code Review Methodology

Following the behavioral tests, we reviewed the implementation with emphasis on cryptographic design, state consistency, log integrity enforcement, and malformed-input handling.

- **Log format:** The log file consists of a plaintext salt header followed by encrypted metadata and append-only encrypted entries.
- **Key derivation and authentication:** Authentication tokens are converted into AES keys using PBKDF2-HMAC-SHA256.

- **Encryption design:** Entries are encrypted using AES-GCM with per-entry IVs and authentication tags.
- **Integrity checks:** The implementation uses authenticated encryption and chaining metadata to detect malformed or reordered records.
- **Input validation:** Names, timestamps, room identifiers, duplicate flags, and gallery-state transitions are validated before committing new entries.

5.6 Attack Summary

ID	Type	Status	Impact
G21-B1	Correctness	Reproduced	Duplicate rooms in `-R` history output
G21-B2	Compatibility	Reproduced	Some safe rejections use the wrong return code

Table 3: Summary of Group 21 findings

5.7 Tests Attempted but Found Secure

Besides the behavioral issues listed above, we also tried several attacks that did *not* lead to successful security breaks in Group 21:

- **Invalid state transitions:** Attempts to leave without entering, enter two rooms at the same time, or perform duplicate arrivals were rejected without committing unsafe entries.
- **Wrong-token append attempts:** Appends using an incorrect token did not modify the protected log state.
- **Direct log tampering:** The tested byte-modification scenarios did not produce an accepted corrupted state.
- **Batch invalid-line handling:** Invalid batch lines were handled safely, even when the reported return code did not always match the expected convention.
- **Malformed input arguments:** Invalid names, conflicting flags, and invalid room identifiers were rejected before affecting the log.

5.8 Conclusion for Group 21

Group 21 passed most of the shared behavioral suite. The main observed problems were not successful security attacks: room-history output duplicates revisited rooms, and some invalid operations are signalled with an incorrect return code even though the actions are safely rejected. Because the unsafe actions were not committed, we do not classify those return-code mismatches as integrity violations.

6 Implementation G1: Group 1

6.1 Target Summary

- **Assigned implementation:** Group 1.
- **Language and build system:** Java 17 with Maven.
- **Programs analyzed:** `logappend` and `logread`.
- **Analysis status:** Successful vulnerability identified and reproduced.
- **Main result:** Behavioral testing and code review revealed that the implementation reuses the same AES-GCM IV for all records in a log file, causing a catastrophic confidentiality failure and enabling cryptographic attacks against the encrypted log structure.

6.2 How We Built and Ran the Implementation

```
cd SRSD_GalleryLog
mvn package

./logappend -T 1 -K segredo -A -E Alice galeria.log
./logappend -T 2 -K segredo -A -E Alice -R 5 galeria.log

./logread -K segredo -S galeria.log
```

Observed result: The implementation compiled successfully using Maven and generated two executable wrapper scripts, `logappend` and `logread`, which invoke the packaged JAR files. Basic functionality tests succeeded, including append operations, room transitions, history queries, integrity verification, and batch mode execution.

6.3 Behavioral Test Results

Table 4 summarizes the Group 1 execution of the shared test suite described in Section 4. As with Group 21, the full transcript is omitted from the report and submitted separately as an artifact.

Test area	Result	Relevant observation
Basic gallery flow	Pass	Normal append and state-query behavior worked as expected.
Room transitions	Pass	Room entry and exit transitions were enforced correctly.
State and intersection queries	Pass	`-S` and `-I` matched the expected scenarios.
Room history query	Minor issue	Revisited rooms were printed again, e.g., `3,7,3` instead of `3,7`.
Invalid actions	Pass	Invalid transitions were rejected without corrupting the log.
Wrong token and tampering	Pass	Wrong-token and tampered-log tests were rejected safely.
Batch mode	Issue	Batch handling deviated from the expected behavior in the tested invalid-line scenario.

Table 4: Condensed behavioral results for Group 1

6.4 Behavioral Findings

6.4.1 Room History Duplication

Group 1 shows the same `-R` behavior as Group 21: repeated visits to the same room are printed multiple times. For example, a visit sequence through rooms 3, 7, and 3 is reported as `3,7,3`, while the expected output is `3,7` if room history is interpreted as the list of distinct rooms visited in first-visit order. This is a correctness issue and not, by itself, a confidentiality or integrity break.

6.4.2 Batch Mode Deviation

The shared demo also exposed a batch-mode issue. The expected behavior is that valid batch lines are processed safely and invalid lines are rejected according to the project specification, without leaving the log in an inconsistent or partially unsafe state. Group 1 deviated from this expected behavior in the tested batch case. We therefore record it as a behavioral issue, separate from the cryptographic vulnerability discussed below.

6.5 Reconnaissance and Code Review Methodology

We reviewed the implementation with the security goals from the project statement in mind: confidentiality of the log contents without the token, integrity against unauthorized modification, authenticity of appends and reads, and correct handling of invalid gallery-state transitions.

- **Log format:** Each log file begins with a fixed header containing a magic value (BLOG), a version byte, and a random salt. Each appended record stores:

  ```
  length || IV || AES-GCM(ciphertext)
  ```

  The plaintext encrypted inside each record contains:

```
prevHash || timestamp || payload
```

where `prevHash` is the SHA-256 hash of the previous raw record.

- **Key derivation and authentication:** The implementation derives a 256-bit AES key from the provided token using PBKDF2-HMAC-SHA256 with 120000 iterations and a per-log random salt. AES-GCM authentication tags are used to verify ciphertext integrity.
- **Encryption design:** The implementation uses AES-256-GCM for authenticated encryption. However, the IV generation is critically flawed. The IV is deterministically generated from the log filename using:

```
SecureRandom vulnerableRandom = new SecureRandom();
vulnerableRandom.setSeed(logPath.getBytes(...));
vulnerableRandom.nextBytes(iv);
```

  As a result, every record appended to the same log file reuses the exact same AES-GCM IV under the same encryption key. AES-GCM nonce reuse is cryptographically catastrophic and breaks confidentiality guarantees.
- **Integrity checks:** The implementation validates a SHA-256 hash chain linking consecutive records. Any modification, deletion, truncation, or byte tampering inside a record causes AES-GCM authentication failure or hash-chain mismatch detection.
- **Input validation:** The implementation validates timestamps, room identifiers, state transitions, duplicate flags, malformed arguments, and ordering constraints using a dedicated state machine.

6.6 Attack Summary

ID	Type	Status	Impact
G1-A1	Confidentiality	Exploited	AES-GCM nonce reuse leaks plaintext relationships
G1-A2	Integrity	Not fully exploited	Potential forgery conditions due to nonce reuse

Table 5: Summary of findings for Group 1

6.7 Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation

6.7.1 High-Level Description

The implementation deterministically derives the AES-GCM IV from the log filename instead of generating a fresh random nonce for every encrypted record. Consequently, all records stored in the same log file reuse the same IV under the same AES key.

AES-GCM requires nonce uniqueness for security. Reusing a nonce with the same key completely breaks confidentiality and may enable forgery attacks. An attacker without the token can compare ciphertexts from different records and derive relationships between plaintext contents.

6.7.2 Root Cause

The vulnerability originates in the `CryptoService.randomIv()` method:

```
static byte[] randomIv(String logPath) {
    byte[] iv = new byte[Constants.GCM_IV_BYTES];
    SecureRandom vulnerableRandom = new SecureRandom();
    vulnerableRandom.setSeed(logPath.getBytes(StandardCharsets.UTF_8));
    vulnerableRandom.nextBytes(iv);
    return iv;
}
```

Because the seed depends only on the log path, every append operation targeting the same file generates the same IV. The resulting AES-GCM nonce reuse violates the security assumptions of GCM mode.

6.7.3 Exploit Preconditions

- The attacker can access the encrypted log file but does not know the token.
- The attacker can observe multiple encrypted records stored in the same log.
- The attacker does not need to modify the file or interact with the legitimate user.

6.7.4 Reproduction Steps

```
# 1. Create a log with multiple records
rm -f nonceReuse.log

./logappend -T 1 -K secret -A -E Alice nonceReuse.log
./logappend -T 2 -K secret -A -G Bob nonceReuse.log
./logappend -T 3 -K secret -A -E Alice -R 1 nonceReuse.log

# 2. Observe the binary log contents
xxd nonceReuse.log

# 3. Compare encrypted records
# The IV prefix of every encrypted record is identical.
```

6.7.5 Expected Output on a Correct Implementation

```
Each encrypted record should use a unique random IV.
```

6.7.6 Observed Output on Group 1

```
The IV bytes at the beginning of each encrypted record are identical across
all records stored in the same log file.
```

6.7.7 Security Impact

The attack is security-relevant because AES-GCM nonce reuse completely invalidates the confidentiality guarantees of the encrypted log. Since multiple plaintexts are encrypted using the same keystream, an attacker can compute XOR relationships between plaintext records without knowing the encryption key.

The plaintext structure is highly predictable because every record contains:

- a fixed-size previous hash field,
- a timestamp,
- structured payload fields,
- repeated names and room identifiers.

This predictability enables statistical plaintext recovery and cryptographic analysis of encrypted records. Under certain conditions, AES-GCM nonce reuse may also enable message forgery attacks.

6.7.8 Attack Code

The attack requires no token knowledge and no custom exploit code. The vulnerability can be demonstrated directly by examining multiple encrypted records inside the same log file using:

```
xxd nonceReuse.log
```

6.8 Finding G1-A2: Potential Integrity Forgery via AES-GCM Nonce Reuse

6.8.1 Description

During the analysis, we identified that the AES-GCM nonce reuse vulnerability may also enable integrity attacks or ciphertext forgery attempts. Although we did not complete a practical forgery exploit during the Break-it phase, the cryptographic misuse significantly weakens the authenticated encryption guarantees of AES-GCM.

Because the implementation correctly validates hash chains and authentication tags, we were unable to produce a complete unauthorized append accepted by `logread`. Nevertheless, nonce reuse under AES-GCM is considered a severe cryptographic failure.

6.9 Tests Attempted but Found Secure

We also tested several attack ideas against Group 1 that did not result in successful exploits:

- **Record reordering attack:** Reordering existing records broke the previous-record hash chain and was rejected.
- **Direct ciphertext tampering:** Modifying bytes inside existing records caused AES-GCM authentication or chain validation to fail.

- **Wrong-token operations:** Reads and appends with an incorrect token were rejected safely in the tested cases.
- **Invalid gallery transitions:** Duplicate arrivals, leaving without entering, and invalid room transitions were rejected by the state machine.
- **Malformed length fields:** Corrupted record length fields did not lead to an exploitable crash or uncontrolled memory allocation.

6.10 Vulnerabilities Considered but Not Exploited

- **Record reordering attack:** We tested whether records could be reordered while preserving the hash chain. The implementation correctly rejected reordered logs due to previous-record hash validation.
- **Malformed length fields:** We investigated whether corrupted record length fields could trigger parser crashes or uncontrolled memory allocation. The implementation included explicit upper bounds on record size and rejected malformed lengths safely.

6.11 Conclusion for Group 1

The strongest finding against Group 1 was a catastrophic AES-GCM nonce reuse vulnerability caused by deterministic IV generation based solely on the log filename. Although the implementation correctly enforced authentication, state validation, and hash-chain integrity, the cryptographic misuse fundamentally broke confidentiality guarantees for encrypted records.

Overall, the implementation demonstrated good defensive programming practices in several areas, but the IV generation flaw constitutes a severe cryptographic design error.

7 Submitted Attack Artifacts

The following files should be submitted together with this report if attacks are implemented:

- `attacks/group1/`: **[Scripts, modified logs, and README/testcase commands for Group 1.]**
- `attacks/group21/`: **[Scripts, modified logs, and README/testcase commands for Group 21.]**

8 Overall Conclusion

Both implementations passed most of the shared behavioral suite. The common behavioral issue was room-history duplication in exttt-R: revisited rooms were printed multiple times instead of being deduplicated. Group 1 additionally showed a batch-mode deviation, while Group 21 sometimes signalled safe rejections with an incorrect return code. We classify these demo findings as correctness or compatibility issues unless the unsafe action is actually committed or protected data is exposed. Separately, Group 1's deterministic AES-GCM IV generation remains the strongest security-relevant cryptographic finding in this report.